After the ICMLA rejection: what we need to address

Alessio Brini

July 16, 2026

Summary. The reviews are fair and the items you picked out are the right ones. I went through the code against them before writing this, and the situation is worse than the reviews suggest: the two headline results do not currently measure what we say they measure, and there is a unit bug in the fee model that no reviewer could have caught from the manuscript. This is a rebuild of the empirical section rather than a revision of it. The framing, the formulation, and the protocol all survive. The numbers do not.
Every claim below cites a file and a line so you can check it yourself rather than take my word for it.

---

1. Where we stand

Four reviewers: #5 Weak Accept, #6 Weak Accept, #7 Reject, #8 Weak Reject. We were close, and the rejection was methodological rather than about the idea. Clarity scored Very Good, Very Good, Excellent, and Fair. Reviewer #5 called the analysis of the results great, the competitor set good, and the rolling evaluation "far stronger than a single random split of a financial series." Reviewer #6 rated originality and significance Very Good. The reward design and the fee-tier interpretation were both named as strengths. None of that is in question.

Your three items were the right three. Two of them are worse than the reviewers could see:

- **Pseudoreplication in the significance tests.** Confirmed, and slightly worse.
- **Training overlap in the transfer same-period split.** Confirmed, and much worse. The 88.5% number does not depend on the transfer at all.
- **Baseline comparison.** Confirmed, and it interacts with a tuning leak that decides the paper's main result.

On the "try more than PPO" critique: you are right that this should not sink a paper on its own, and only two reviewers raised it as an add-on rather than a reason to reject. But it is cheap insurance, and it is now partly forced. Once the tuning objective is fixed, PPO's margin moves, and a second algorithm turns "PPO against heuristics" into "learned control against heuristics," which is a stronger claim and answers Reviewers #6, #7, and #8 at once.

2. What the audit found

Ordered by how much damage each item does.

2.1 The headline PPO result is tuned on the test set

In `rl-code/baseline_comparison_statistical.py:167`, the Optuna objective evaluates on `test_env`, which is built from the held-out window:

```
# Evaluate on test set
test_reward = evaluate_ppo_model(model, Monitor(test_env))
return test_reward, model
```

`train_optimized_ppo` (lines 199 to 208) returns `best_reward`, the maximum over 10 trials of that test reward, and line 361 records it as the reported out-of-sample number. So every PPO figure in the paper is a max-over-10 order statistic computed on the evaluation data. There is no validation split anywhere in the pipeline. Optuna also selects the action grid (`config/uniswap_rl_param_1108.yaml:12`), so the agent chooses its own action space by test reward. Reviewer #5 suspected exactly this and was right.

The leak is asymmetric, in the direction that produces our story. The competitors are also argmax'd on test, so the leak is at least mutual, but the budgets are not. `ILMinimizer` has zero free parameters (`baseline_strategies.py`, lines 107 to 124). `PassiveWidthSweep` filters its width candidates to those present in `env.action_values` (lines 44 to 45), and on WETH the action values are `[0, 45, 50, 55]` while the candidates are `range(20, 201, 10)`. The intersection is the single width `[50]`. On WETH, PassiveWidthSweep sweeps nothing: it is a fixed-width-50 strategy. PPO gets 10 trials plus its choice of action grid; the baselines get one point or none.

On WBTC, where the grid `[0, 60, 120, 180]` leaves three widths for the sweep, PPO loses. The pool where the baselines get more tuning budget is the pool where PPO underperforms. That pattern is consistent with the results measuring tuning budget rather than learning, and as the paper stands we cannot rule that out.

2.2 The 88.5% transfer number does not depend on the transfer

From `rl-code/cross_pool_transfer.py`, lines 158 to 173:

```
train_split       = int(max_hours * 0.70)
same_period_split = int(max_hours * 0.85)

weth_train       = weth_data.iloc[:train_split]                    # 0 to 70%
weth_same_test   = weth_data.iloc[:same_period_split]              # 0 to 85%
weth_future_test = weth_data.iloc[train_split:same_period_split]   # 70 to 85%
```

Three problems compound here:

- The same-period test (0 to 85%) is a **superset of the training set** (0 to 70%). Roughly 82% of the "test" is training data. The script's own print statement says so: `"Same-period test: 0 - {n} hours (overlaps with training)"`.
- It is also a superset of the future test (70 to 85%), so the same-versus-future comparison pits a set against its own subset.
- The reward is an episode **sum** (lines 118 to 126, `total_reward += reward`), and the same-period segment is 5.67 times longer than the future segment. "Same beats future" is therefore guaranteed by episode length alone, with or without any transfer effect. That is what H1 and H2 actually measure.

The decisive evidence is in the saved output. In the transfer results file `transfer_learning_detailed.csv`, values repeat bit-for-bit **across different training sources**. On row 1, `WETH->WBTC_same` and `WBTC->WBTC_same` are both `141125.41904433558`. The value `131484.1968564255` appears as both `WETH->WETH_same` and `WBTC->WETH_same`. The evaluation outcome does not depend on which pool the policy trained on, which is the signature of a deterministic policy collapsed to a constant action.

I recomputed the column means from that file:

Column	Mean
`WETH->WBTC_same`	102,627.09
`WBTC->WBTC_same`	115,924.94

102,627/115,925 = 88.5%. That is the number in our abstract, and it is a ratio of two policy-independent quantities drawn from columns that share identical entries.

2.3 A fee unit bug of roughly 100 times

`delta` carries two contradictory unit conventions inside the same file:

- `custom_env.py`, lines 494 to 502, `_fee_to_tickspacing`, maps `0.05` to tick spacing 10 and `0.30` to 60. Those are the canonical Uniswap v3 tick spacings for the 0.05% and 0.30% tiers. Here `delta` means *percent*.
- `custom_env.py`, lines 504 to 511, `_calculate_fee`, computes `(delta / (1 - delta)) * l * (...)`, using `delta` directly as a *fraction*. Here `0.05` means 5%.

The configs set `delta: 0.05` (WETH) and `delta: 0.30` (WBTC). The tick-spacing function is the correct one, so the fee function inflates fee income by about 100 times. Our simulated pools charge 5% and 30%, not 0.05% and 0.30%.

The magnitudes corroborate this. The position is 10 ETH (`x: 10`, roughly $30k over the sample), yet episode rewards reach about $141k, several times the capital in 1,500 hours. That is not an LP return. It is what a 5% fee tier pays.

This is the item with the longest reach. Our central economic claim, that the fee tier sets the reward ceiling and that the active-learning advantage is fee-tier dependent, is currently a comparison between a fictional 5% pool and a fictional 30% pool. It may well survive re-running at the true tiers, but as it stands it is not evidence about Uniswap. It is also the most likely cause of the constant-action collapse in the transfer output: at a 5% fee, fee income swamps impermanent loss and gas, so "never rebalance, just collect" sits close to optimal.

2.4 Pseudoreplication

`baseline_comparison_statistical.py`, lines 386 to 389, replicates each deterministic baseline's single window value five times to match the seed count:

```
# Add to all_results replicated across all seeds
# (baselines are deterministic, so same value for all seeds)
for _ in range(n_seeds):
    all_results[name].append(baseline_reward)
```

The test at line 249 is `stats.ttest_ind`, which is unpaired and assumes equal variance, applied to data that is paired by window. So $N = 50$ is 10 distinct baseline numbers each counted five times, and the test sees about 98 degrees of freedom where the baseline carries about 9. There is no multiple-comparison correction across four competitors and two pools. Separately, line 250 computes `wins = np.sum(baseline_data > rewards_arr)` where `baseline_data` is PPO, so the `Wins_vs_PPO` column counts PPO's wins rather than the competitor's.

The good news: `rl-code/analysis_utils.py`, lines 74 to 148, already implements a paired path that reads straight off the saved CSVs, and `paper_figure_pipeline.py`, lines 229 to 239, already recovers window identity through `row_index // 5`. This fix is post-processing with no retraining. Its p-value uses a normal approximation and needs a t-distribution at $n = 10$.

2.5 The paper overstates training by 10 times

`IEEE_main.tex:173` states 1,000,000 environment timesteps per window. Both training scripts pass `params["total_timesteps_1"]` (`baseline_comparison_statistical.py:164`, `cross_pool_transfer.py:99`), and every config sets `total_timesteps_1: 100000`. The unused `total_timesteps: 1000000` sits next to it. Actual training is 100k steps.

The same line says "10 trials per window," but the loop runs 10 trials *per seed* (lines 297 and 352 to 360), so 50 PPO trainings per window and 500 per pool.

2.6 Simulator gaps

These answer Reviewer #5's question (5), and the honest answers are not good:

- **No volume data exists.** `data_price_uni_h_time.csv` has columns `timestamp,price` and

nothing else. Fees are a deterministic function of hourly price displacement, not of swap-level or aggregate volume. Round-trip trading within an hour earns us nothing.

- **No liquidity share.** The LP is the sole provider in its range (`reset:188`); liquidity comes from our own capital. There is no pool TVL and no dilution, so a 10 ETH position collects the entire range's fees.
- **Recentering does not realize impermanent loss and is not value-conserving.** `step`, lines 311 to 320, re-derives liquidity from `xt` alone and carries `yt` over unchanged, with no swap and no slippage. The out-of-range branch (lines 296 to 303) does `xt = xt/2; yt = xt*pt`, which manufactures the missing side.
- **Forced repositioning is free.** When price exits the range under `action == 0`, the environment repositions but `_indicator` reads the original action, so no gas is charged.
- **Gas is a flat $5** from 2021 to 2025, with no gas series and no dependence on position size or ETH price.

2.7 Reproducibility

The WBTC price file `data_price_wbtc_usdc_030_h_time.csv`, referenced by `cross_pool_transfer.py:42` and `pool_wbtc_usdc_030.yaml:8`, is not in the repo. The WBTC and transfer experiments do not run from a clean clone. The `paper/` directory is gitignored (`.gitignore:7`) and nothing under it is tracked, so the manuscript has no version history, only the hand-kept `CHANGES.md`. No conda environment spec is committed. Finally, `rl-code/output/baseline_comparison_OLD/` holds a prior $N = 50$ run with materially different numbers (PPO mean 24,249 against 37,332 in `_optimized`) and unresolved provenance.

3. What this costs us

Claim	Status
PPO beats 3 of 4 competitors on WETH	Not established. PPO's number is a max-over-10 test-set statistic, and its main competitor sweeps a single width.
PPO underperforms on WBTC, so the advantage is fee-tier dependent	Not established. Confounded with tuning budget, and the fee tiers in the simulator are 5% and 30%, not 0.05% and 0.3%.
88.5% same-period transfer efficiency	Not established. A ratio of two columns holding identical, policy-independent values.

The counterweight is real. The framing, the MDP formulation, the reward design, the writing, and the rolling-window protocol all survive, and reviewers praised each of them. The protocol itself is sound. It is the objective we plugged into it that is wrong. The scaffolding stands; the numbers do not.

4. The work list

Ordered by dependency rather than effort. P0 and P1 change every number in the paper, so no compute gets spent before they land.

P0. Settle the fee units.

Fix `_calculate_fee` to take a fraction (0.0005, 0.003), keep `_fee_to_tickspacing` consistent with it, and add a test pinning a known fee tier to a plausible return. Resolve the recentering value leak at the same time, since it also changes every number. Hours, not days. Everything waits on this.

P1. Kill the tuning leak.
Split the 7,500-hour training block into training plus validation (the natural choice: last 1,500 hours as validation, 6,000 for training). Point the Optuna objective at validation and never at `test_df`. Give the competitors a matched budget on the same validation segment, and fix `PassiveWidthSweep`'s candidate filter so it sweeps a real range instead of collapsing to `[50]`. Touch test exactly once, after selection. This replaces every PPO number: 500 trainings for WETH, 1,100 for WBTC. It is the expensive step and the unavoidable one. If a referee finds this independently, no other fix saves the paper.

P2. Window-level paired statistics.
Average PPO's seeds within a window, keep one baseline value per window, and run a paired test or a Wilcoxon signed-rank test, given the heavy tails that both #5 and #8 flagged, on $n = 10$ for WETH and $n = 22$ for WBTC. Add seeded bootstrap confidence intervals (the current bootstrap is unseeded, so our CIs are not reproducible) and a correction across the competitor family. Reuse `analysis_utils.py`. Report seed dispersion as a variability statistic, not as sample size. This is your first item, and it is nearly free.

P3. Rebuild transfer under the main protocol.
Exactly what Reviewer #8 asked for, and what our own line 297 already promises as the natural next extension. Drop the 0 to 85% split, report mean per-step reward rather than episode sums so unequal-length segments compare, and use the last 15% of history that no current split touches. Diagnose the constant-action collapse first, and recheck after P0, since it is probably downstream of the fee bug. Restore the missing WBTC CSV before anything here runs.

P4. Baselines.
Add buy-and-hold and full-range liquidity. This is Reviewer #5's sharpest point: our reward is defined as impermanent loss relative to holding, yet hold is not a baseline, so a policy can beat every LP strategy while still losing to simply keeping the tokens. Note that buy-and-hold cannot go through `env.step()`, because the environment force-repositions when price exits the range (lines 274 to 306), so compute $W_T = x_0 p_T + y_0$ directly or add a passthrough mode. About 25 lines plus a tuple in `baseline_configs` (lines 301 to 325) and labels in `analysis_utils.py`, lines 13 to 28. A deterministic hold baseline inherits the five-times replication bug unless P2 lands first.

P5. A second learned algorithm.
A2C and DQN are drop-in on the existing `Discrete` action space (`custom_env.py:94`): add an `ALGOS` dict, a per-algorithm kwarg filter, and split the PPO-specific YAML block (`clip_range`, `target_kl`, `gae_lambda`, `vf_coef`). SAC and TD3 need a `Box` action space and a real refactor, worth it only if we move to continuous width control, which would incidentally kill the "action grid was tuned" objection outright.

P6. Write-up, last.
Do not start until P1 through P3 produce numbers. `CHANGES.md` shows 11 sentence-level passes already spent on prose that may not survive. Then: disclose the split in the abstract rather than in footnote 3 on page 5, since that asymmetry is most of why three reviewers independently flagged it; correct the timesteps and the trial count; and take Reviewer #8's structural asks, which are free ("Related Works" to "Related Work," label Section III as background and Section IV as method, split the conclusion into discussion, limitations, and conclusion, cite the comparator strategies or state that they are ours, and cite the RL transfer-learning literature). Document the simulator honestly. Twenty references is thin.

Prerequisites, not chores: commit the WBTC CSV, commit an `environment.yml` for a dedicated conda environment, and decide whether to track `paper/` so the manuscript has version history.